Introduction to Statistics in R

June 11, 2026

CCM

Data Visualization and Statistics in R: Boot Camp

Hands-on tutorial to help build skills required in Precision Child Health, data science and bioinformatics for SickKids staff and trainees.

Topics include:

- Data exploration and cleaning
- Data visualization
- Correlations, t-tests, and linear regressions
- Generalized linear models
- Multi-level modeling

Prerequisites: familiarity with RStudio

To register please RSVP at:
https://ccm20260611.eventbrite.com/

Hands-on Bioinformatics Tutorials for Biologists

Thursday June 11, 2026
9 AM – 12 PM

In Multimedia Room, PGCRL 3rd floor, or on Zoom

Note that prior knowledge of R is recommended to get the most out of this workshop.

Check out other workshop series hosted by CCM at https://ccm.sickkids.ca/bioinformatics-training/

Organized by the Centre for Computational Medicine (ccm.sickkids.ca) at the SickKids Research Institute with support from other groups:

SickKids Research Training Centre — www.sickkids.ca/research

Digital Research Alliance of Canada — alliancecan.ca

Compute Ontario — computeontario.ca

Table of contents

01 **Data exploration and cleaning**
Overview of the Framingham heart dataset, basic data frame manipulation.

02 **Data visualization**
Plotting with base R vs ggplot2.

03 **Correlations and t-tests**

04 **Linear regressions and contrasts**

05 **Generalized linear models**
Logistic, Poisson, and Zero-Inflated Poisson regression

06 **Multi-level models**

01

Data exploration and cleaning

Framingham Heart Study

One of the longest prospective epidemiological studies of cardiovascular disease and its risk factors.

Began in 1948 in Framingham, MA.

- Initially enrolled 5209 men and women aged 29-62 years old.
- Followed them over time, with assessments every 2 years.
- Data collection for the original cohort ended in 2014.
- Over time, the researchers incorporated offspring and their spouses into the study.
- Today, they have data from three generations of participants as well as more ethnically diverse cohorts.

Framingham Heart Study

Examinations included:

- Detailed medical history
- Physical exams
- Lab tests
- Lifestyle and habits
- Noninvasive imaging

Our dataset: subset of the FHS from Kaggle

- 4000+ records and 16 variables.

(https://www.kaggle.com/datasets/captainozlem/framingham-chd-preprocessed-data)

Variable	Description	Class/type
male	Sex (male or female)	Binary
age	Age	Continuous
education	Education (0-11 years, HS or GED, some uni, uni grad+)	Ordinal/categorical
currentSmoker	Whether the patient is currently a smoker	Binary
cigsPerDay	Number of cigarettes smoked per day, on average	Continuous
BPMeds	Whether the patient is on blood pressure medication	Binary
prevalentStroke	Whether the patient previously had a stroke	Binary
prevalentHyp	Whether the patient is hypertensive	Binary
diabetes	Whether the patient has diabetes	Binary
totChol	Total cholesterol level	Continuous
sysBP	Systolic blood pressure	Continuous
diaBP	Diastolic blood pressure	Continuous
BMI	Body mass index	Continuous
heartRate	Heart rate	Continuous
glucose	Glucose level	Continuous
TenYearCHD	10-year risk of coronary heart disease	Binary

Data exploration

Some common and/or important data cleaning steps include making sure that:

1. All variables are in the right format and/or correctly classified.
2. Duplicate or missing data are dealt with.
3. Variables of interest have been transformed appropriately.
4. Outliers are handled.
5. Data are validated.

Data exploration

1. **Look at the data** and its structure – are all the variables in the right format?
str()

2. **Tabulate data**
table(df$column) or table(df$column1, df$column2, ...)

3. **Modify variables**
mutate(column = function(column))
mutate(df, across(columns, function))

4. **Transform variables** – center, scale, log-transform
log(x, base = exp(1))
scale(x, center = TRUE, scale = TRUE)

Data exploration

5. **Subset dataframes**
filter() rows according to some condition
select() columns to keep
subset(df, subset = row_vals == “abc”, cols = X)

6. **Combine vectors and dataframes**
cbind() to join by columns
rbind() to join by rows
merge() to join dataframes

7. **Reshape dataframes**
pivot_wider() each row is a participant/patient
pivot_longer() each row is an observation, with multiple rows per participant

Data exploration

8. **Handy functions, pipes, and operators**

ifelse() for conditional element selection
case_when() as a vectorized version of multiple ifelse() statements, uses formula notation (more on that later)

%>% passes the result of the function on the left as input to the next function
%in% to compare vectors

Create a summary table

Use summarize() to create a dataframe with summaries of desired variables. For example, what if you're only interested in the educational attainment and smoking habits of people who go on to develop heart disease?

	TenYearCHD	education_class	n_participants	age	currentSmoker	prop_smokers	cigsPerDay
1	0	< High School	1397	51	645	0.46	8.39
2	0	College Graduate	403	47	204	0.51	9.09
3	0	High School Graduate	1106	47	598	0.54	9.67
4	0	Some College	599	48	275	0.46	7.70
5	1	< High School	323	56	160	0.50	9.93
6	1	College Graduate	70	53	36	0.51	11.97
7	1	High School Graduate	147	52	81	0.55	11.14
8	1	Some College	88	53	46	0.52	10.74

02

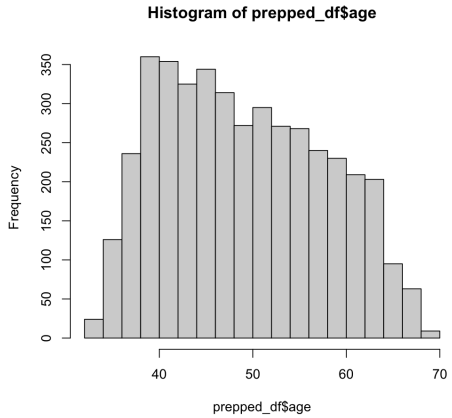
Data visualization

Base R vs ggplot2

Base R

Useful in a pinch but doesn't generate the prettiest figures.

plot() and hist() are useful for taking a quick look at your data.



ggplot2

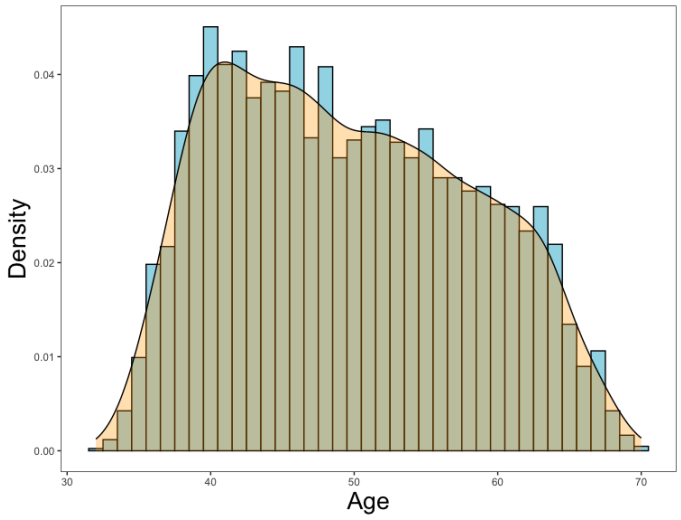
ggplot2 is a package based on the grammar of graphics.

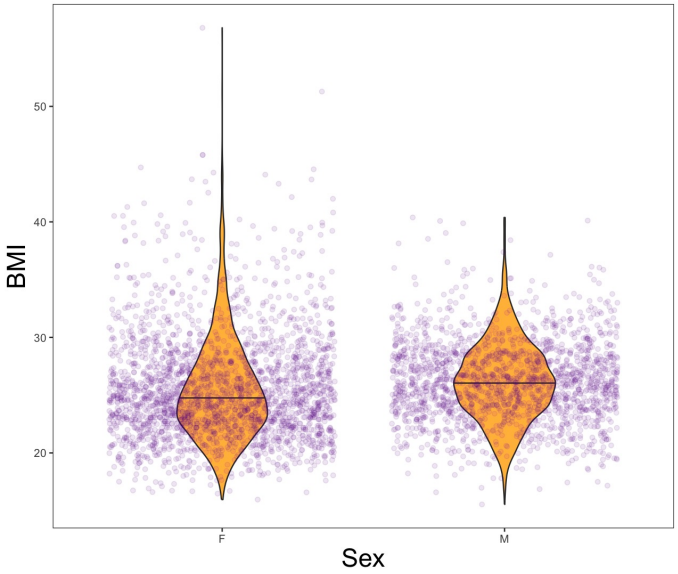
You can create your figure with some basic components:

1. **Data**
2. **Mapping**
3. **Layers/geoms**
4. Facets
5. Statistics
6. Coordinates
7. Theme

Distributions

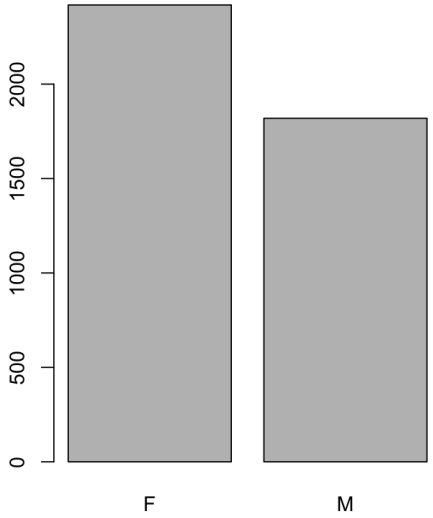
Look at the distribution of data using histograms, density plots, boxplots, and violin plots (among others).



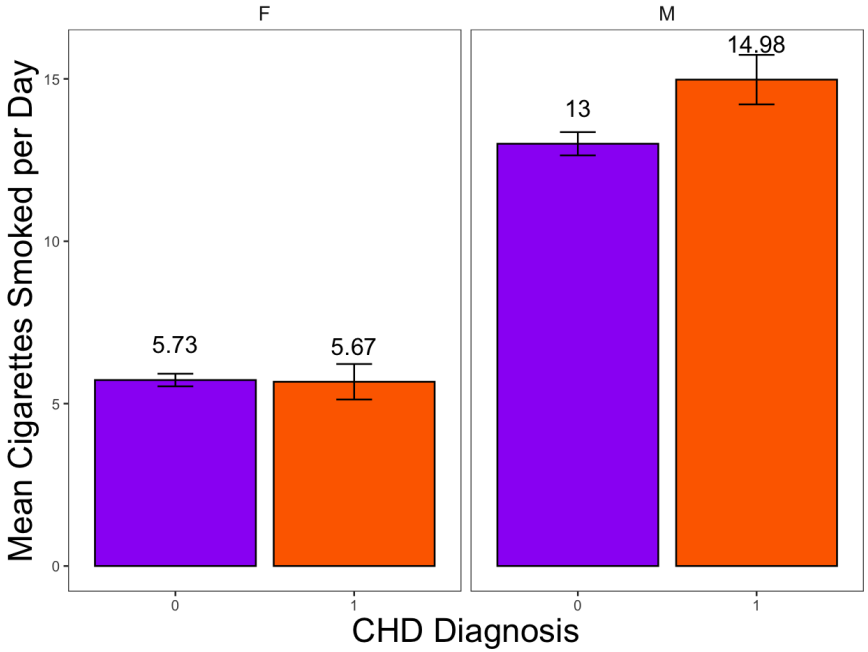


Bar Charts

Compare groups using bar charts.



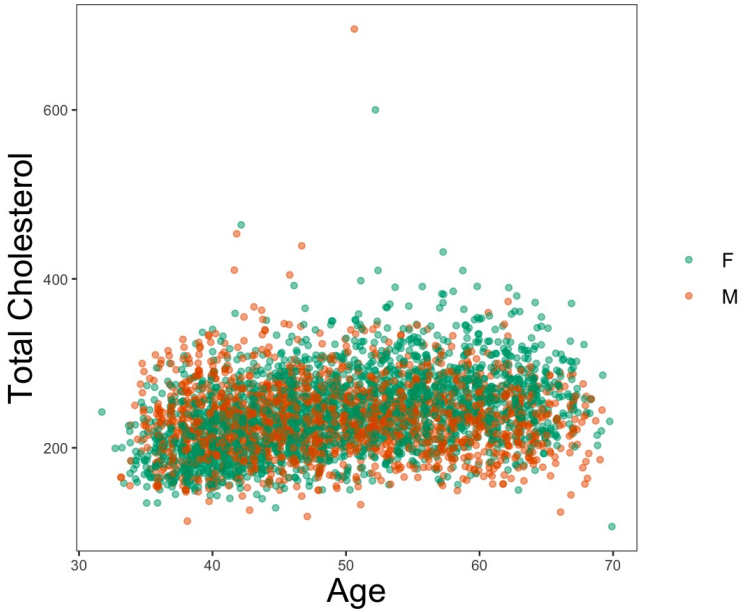
Base R

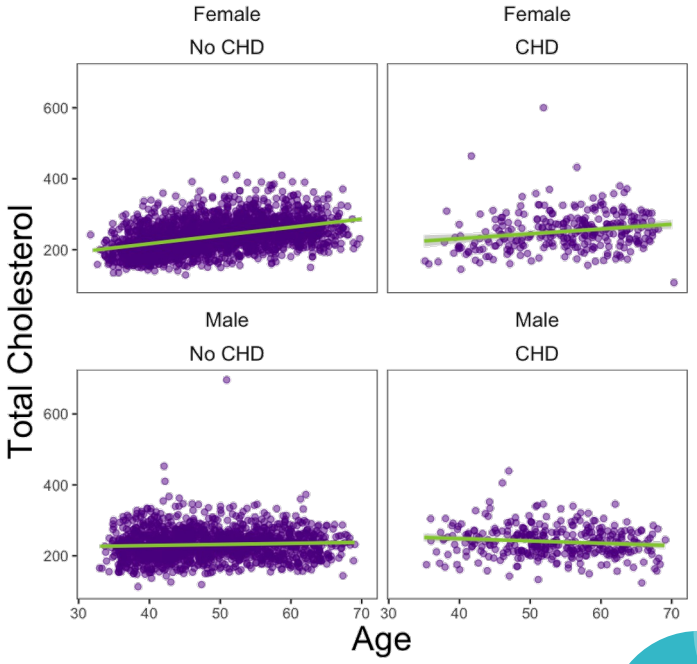


ggplot2

Scatterplots

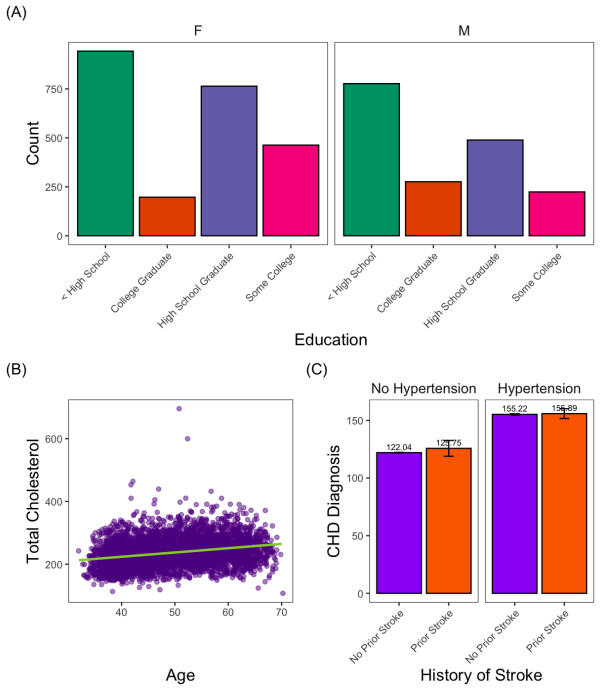
Plot the relationship between two continuous variables using scatterplots.





Panel figures

You might have multiple plots that you want to put into a single figure with panels. You can do this using the patchwork() package.

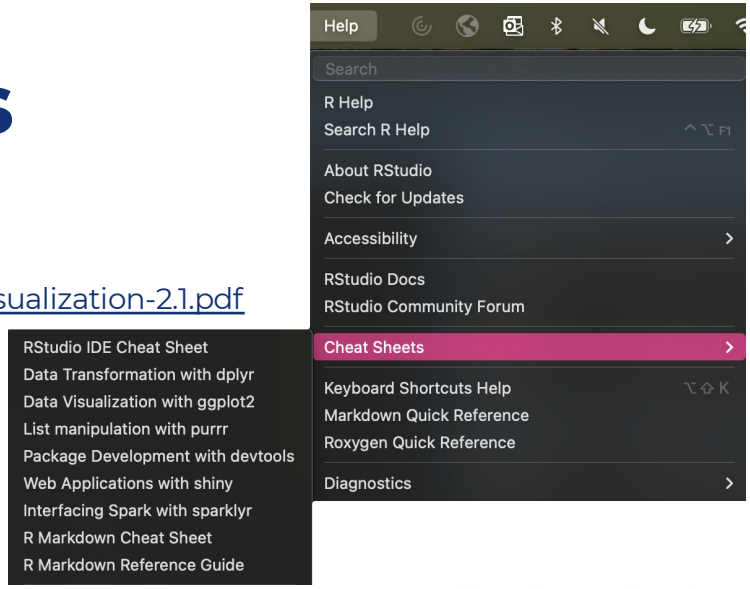


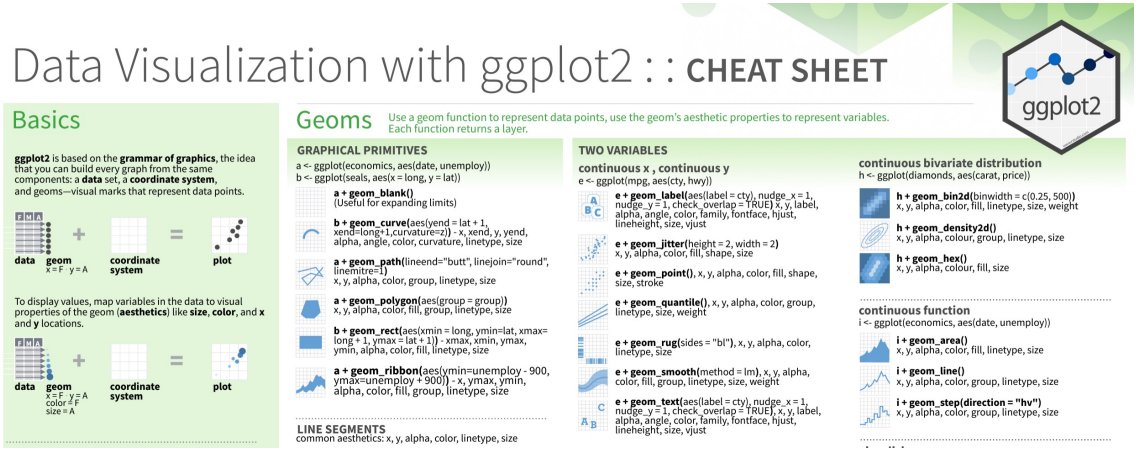
Resources

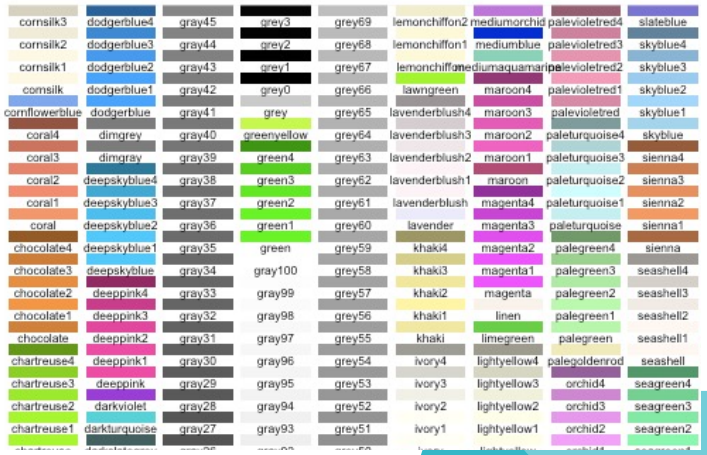
ggplot2 cheat sheet:
https://www.maths.usyd.edu.au/u/UG/SM/STAT3022/r/current/Misc/data-visualization-2.1.pdf

color palette reference:
https://sape.inf.usi.ch/quick-reference/ggplot2/colour

R Studio cheat sheets: Help > Cheat Sheets > ...







03

Correlations and t-tests

Pearson Correlations

A Pearson correlation measures the strength of the linear relationship between two continuous variables.

cor(x, y, use = “complete.obs”, method = “pearson”)

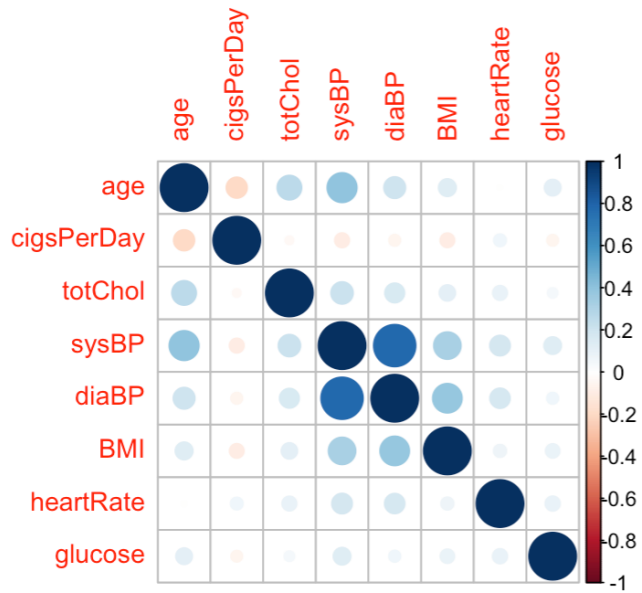
- use = “complete.obs” removes rows where at least one of the values is NA/missing and this is not specified, could also specify “na.or.complete” (output is NA if there are no complete cases)

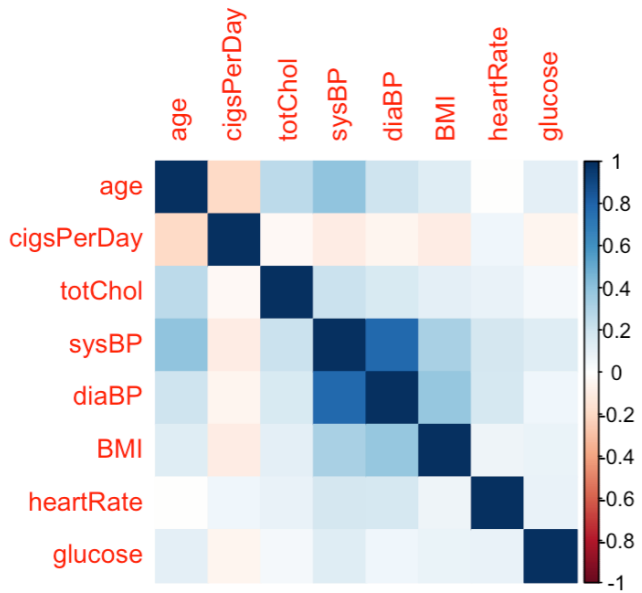
cor.test(x, y, use = “complete.obs”, method = “pearson”)

- provides r, p-value, and confidence intervals.

Correlation Plots

Correlation plots allow you to visualize correlations between all the numeric variables in your dataframe.



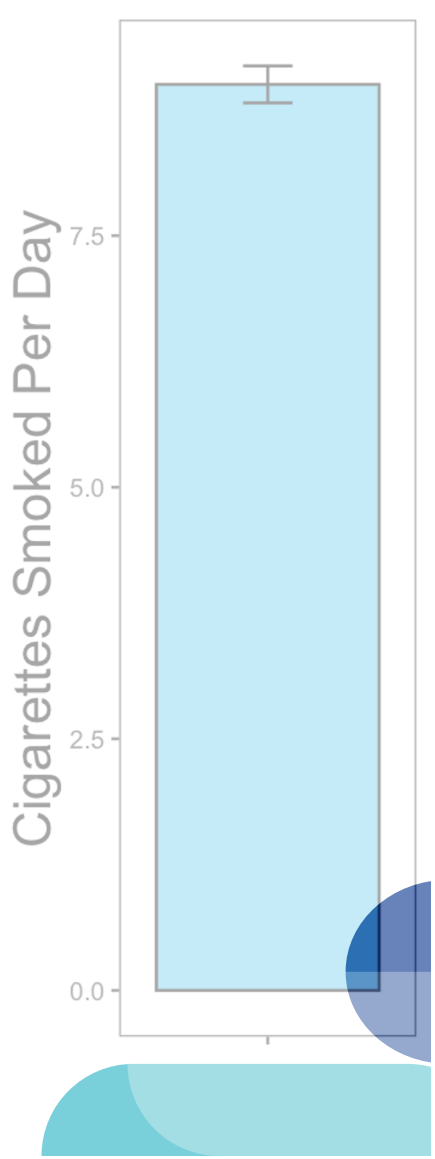


One-way T-tests

Compare your sample mean to the population mean, 0, or another value.

```
t.test(df$outcome, mu = 0)
```

Are the number of cigarettes smoked by our sample significantly different from 0? Different from an average of 10 cigarettes smoked per day?

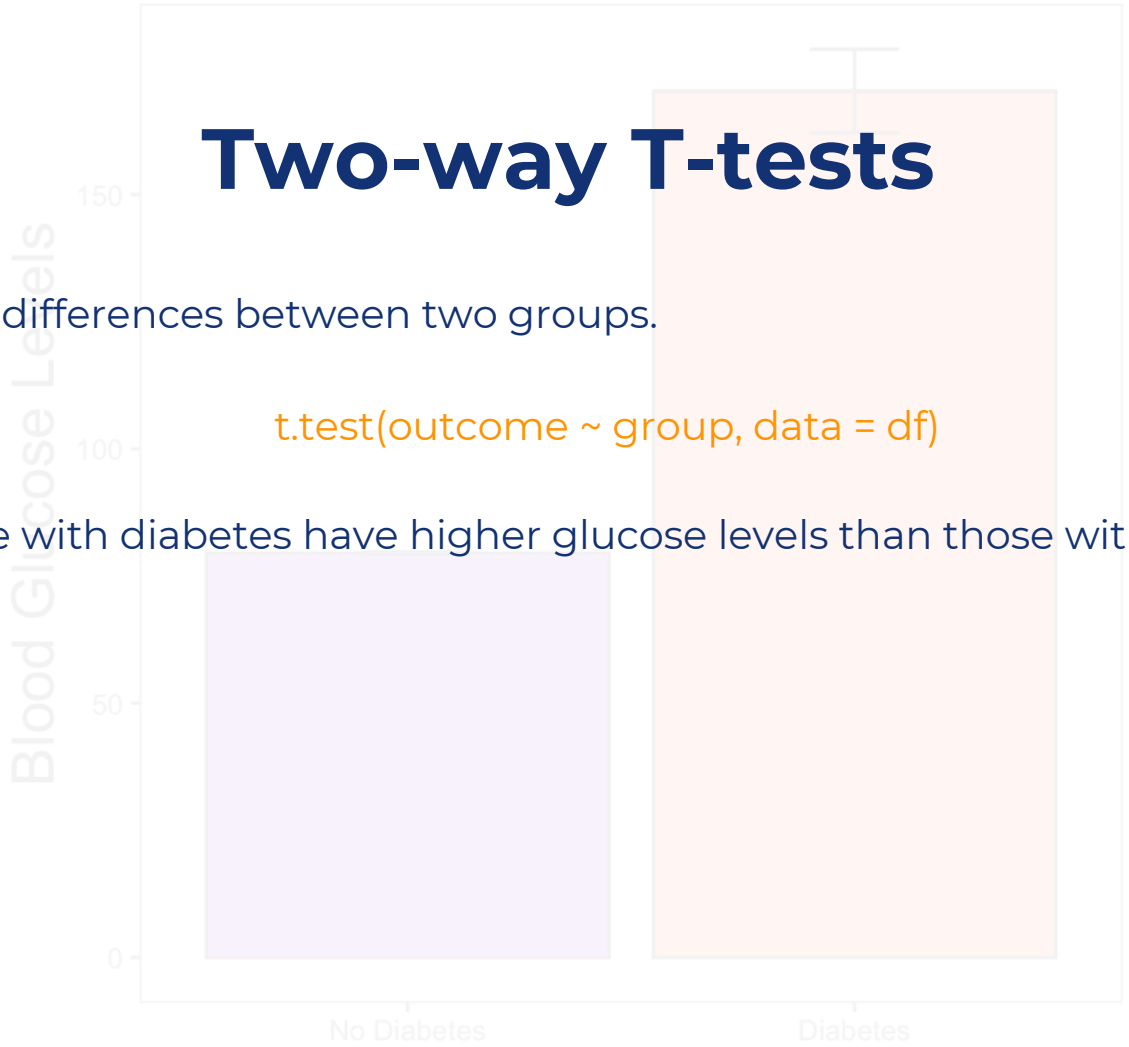


Two-way T-tests

Compare differences between two groups.

```
t.test(outcome ~ group, data = df)
```

Do people with diabetes have higher glucose levels than those without diabetes?



Break

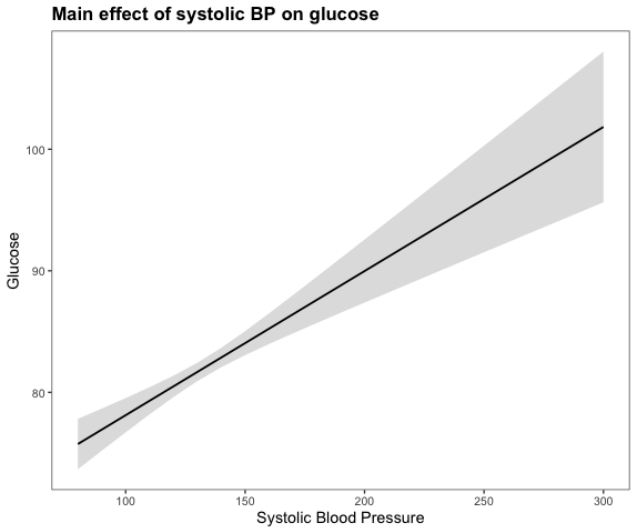
04

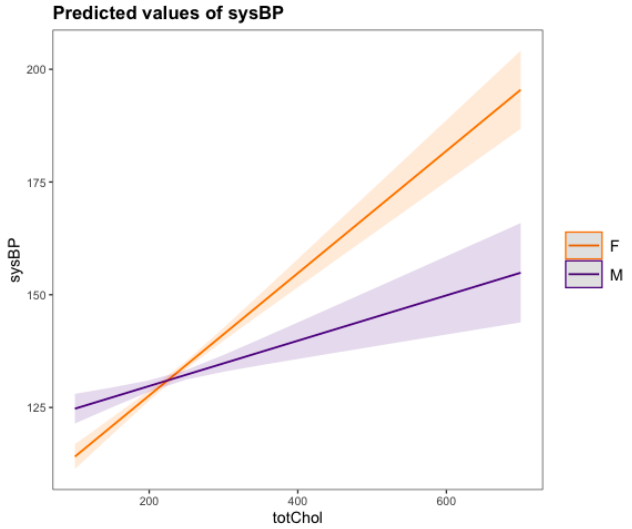
Linear regressions and contrasts

Linear Regression

Model relationships between predictors and continuous outcome variables.

- **Main effects** models reveal the linear relationship between each predictor and the outcome.
- **Interaction effects** reveal how the relationship between a predictor and an outcome is *moderated* by another variable.





Simple Linear Regressions

Formula syntax for conducting linear regressions in R.

Syntax	Description
`outcome ~ predictor1 + predictor2 + ...`	Main effects of predictor 1, predictor 2, etc.
`outcome ~ predictor1 + predictor2 + predictor1:predictor2`	Main effects of predictor 1 and predictor 2, and the interaction between predictor 1 and 2
`outcome ~ predictor1 * predictor2`	Main effects of predictor 1 and predictor 2, and the interaction between predictor 1 and predictor 2
`outcome ~ predictor1 + predictor1:predictor2`	Main effect of predictor 1 only, and the interaction between predictor 1 and predictor 2
`outcome ~ predictor1:predictor2`	The interaction between predictor 1 and predictor 2 only

Main effect models

Examine relationships between predictors and outcomes of all classes.

Are age and blood pressure related to blood glucose levels?

- Both predictors are continuous
- Outcome variable is also continuous

Are smoking status and use of BP medications related to cholesterol levels?

- Both predictors are categorical/binary
- Outcome variable is continuous

Syntax	Description
`outcome ~ predictor1 + predictor2 + …`	Main effects of predictor 1, predictor 2, etc.

Interaction effects

How is the relationship between a predictor and an outcome *moderated* by another variable?

How are sex and cholesterol levels related to blood pressure? Does the relationship between cholesterol levels and blood pressure vary between males and females?

- The predictors are categorical/binary and continuous
- Outcome variable is continuous

Syntax	Description
`outcome ~ predictor1 + predictor2 + predictor1:predictor2`	Main effects of predictor 1 and predictor 2, and the interaction between predictor 1 and 2
`outcome ~ predictor1 * predictor2`	Main effects of predictor 1 and predictor 2, and the interaction between predictor 1 and predictor 2
`outcome ~ predictor1 + predictor1:predictor2`	Main effect of predictor 1 only, and the interaction between predictor 1 and predictor 2
`outcome ~ predictor1:predictor2`	The interaction between predictor 1 and predictor 2 only

Linear Regression

Syntax for a main effects model:

lm(cholesterol_levels ~ sex + BP_meds, data = dataframe)

Syntax for an interaction effect model:

lm(outcome ~ predictor1 * predictor2, data = df)

Syntax	Description
`outcome ~ predictor1 + predictor2 + …`	Main effects of predictor 1, predictor 2, etc.
`outcome ~ predictor1 + predictor2 + predictor1:predictor2`	Main effects of predictor 1 and predictor 2, and the interaction between predictor 1 and 2
`outcome ~ predictor1 * predictor2`	Main effects of predictor 1 and predictor 2, and the interaction between predictor 1 and predictor 2
`outcome ~ predictor1 + predictor1:predictor2`	Main effect of predictor 1 only, and the interaction between predictor 1 and predictor 2
`outcome ~ predictor1:predictor2`	The interaction between predictor 1 and predictor 2 only

Resources

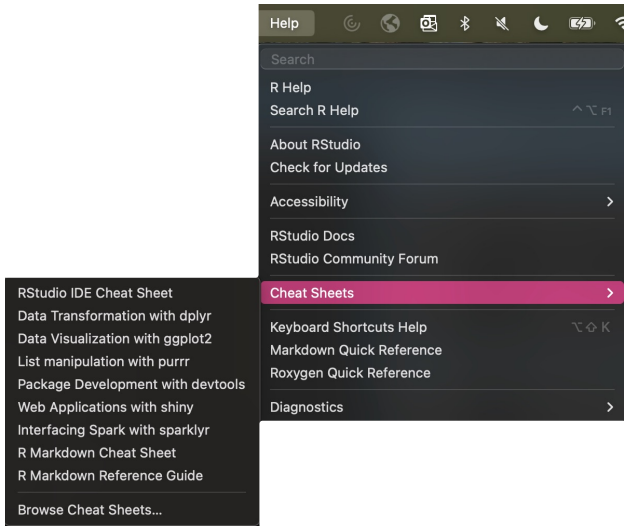
Plotting main effects:

https://cran.r-project.org/web/packages/sjPlot/vignettes/plot_marginal_effects.html

Plotting interactions:

https://cran.r-project.org/web/packages/sjPlot/vignettes/plot_interactions.html

More cheat sheets on R Studio:



Contrasts

Contrasts are linear combinations used to represent categorical predictors in a regression model. For a categorical predictor with *n* levels, you need *n – 1* contrasts to encode the variable.

There are several types of contrasts:

1. Dummy coding
2. Simple effect coding
3. Deviation coding
4. Difference coding
5. Helmert coding

For all contrast types except for dummy coding, the contrast coefficients must sum to 0. This centers the predictor around the mean, making the intercept and coefficients easier to interpret.

Contrasts

By default, R uses **dummy coding** (aka treatment contrasts) where the reference group is the *first level* of the factor (this can be changed) and represented with 0.
The intercept is the mean of the reference group. The resulting coefficients represent the difference between the level and the reference level.

`contr.treatment(n)`

vegetables	Contrast 1	Contrast 2
Never	0	0
Sometimes	1	0
Always	0	1

Gender	Contrast 1
Female	0
Male	1

Contrasts

With **simple effect coding,** you can compare each level to a manually-defined reference level (coded as -1). The intercept represents the grand mean of the outcome variable across all levels, while the coefficients represent the difference between a given level and the reference. Must be specified manually in R!

```
contrasts(df$var) <- cbind(contr1 = c(-1, 1, 0),
                           contr2 = c(-1, 0, 1))
contrasts(df$var) <- c(-1, 1)
```

vegetables	Contrast 1	Contrast 2
Never	-1	-1
Sometimes	1	0
Always	0	1
Sum	0	0

Gender	Contrast 1
Female	-1
Male	1
Sum	0

Contrasts

Deviation coding compares each level to the *grand mean* instead of a reference group. The intercept represents the grand mean while the coefficients represent the difference between the level and the grand mean. There's no reference group.

`contr.sum(n)`

vegetables	Contrast 1	Contrast 2
Never	-0.25	-0.25
Sometimes	0.50	-0.25
Always	-0.25	0.50
Sum	0	0

Contrasts

Difference coding is useful for comparing the mean of the current level to the mean of the previous levels. This is useful when you have ordinal variables that are ordered in a meaningful way (e.g., low, medium, high).

Helmert coding is useful for comparing the mean of the current level to the mean of the *subsequent* levels. Essentially, the opposite of difference coding.

vegetables	Contrast 1	Contrast 2
Never	-1	-0.5
Sometimes	1	-0.5
Always	0	1
Sum	0	0

Difference Coding

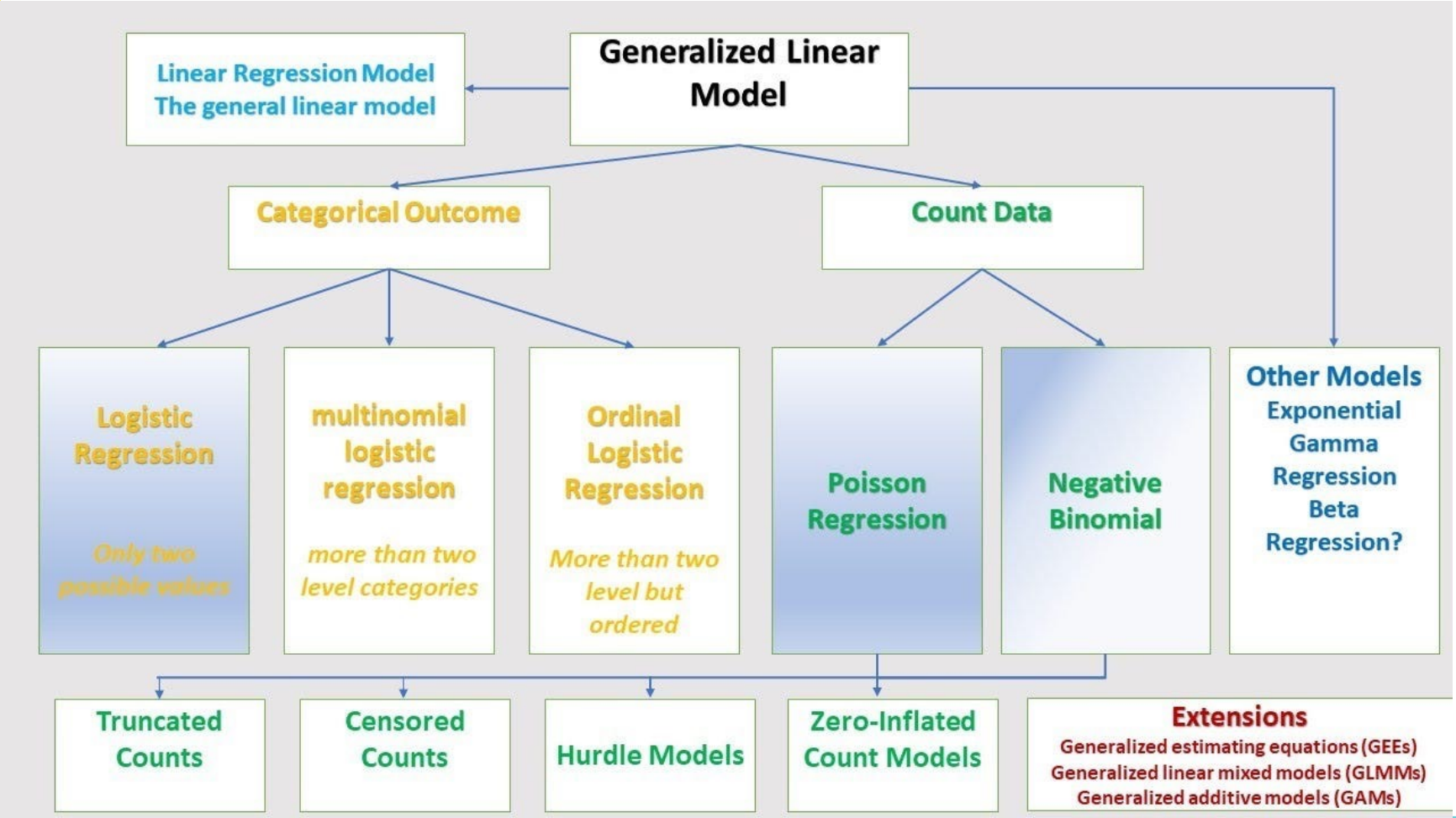
vegetables	Contrast 1	Contrast 2
Never	-1	-1
Sometimes	1	-1
Always	0	2
Sum	0	0

Helmert Coding

Break

05

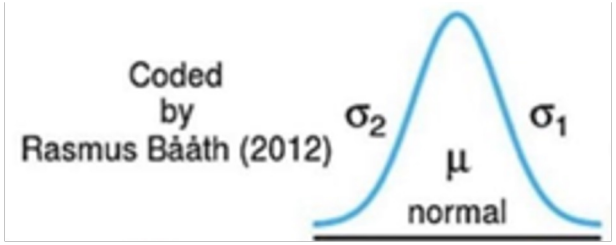
Generalized linear models


Generalized Linear Model
Linear Regression Model
The general linear model
Categorical Outcome
Count Data
Logistic Regression
Only two possible values
multinomial logistic regression
more than two level categories
Ordinal Logistic Regression
More than two level but ordered
Poisson Regression
Negative Binomial
Other Models
Exponential
Gamma
Regression
Beta
Regression?
Truncated Counts
Censored Counts
Hurdle Models
Zero-Inflated Count Models
Extensions
Generalized estimating equations (GEEs)
Generalized linear mixed models (GLMMs)
Generalized additive models (GAMs)

Generalized Linear Models (GLMs)

Linear regressions assume that the residuals of the outcome variable are normally distributed i.e., they follow the Gaussian distribution.

This assumption generally holds well for most continuous variables.



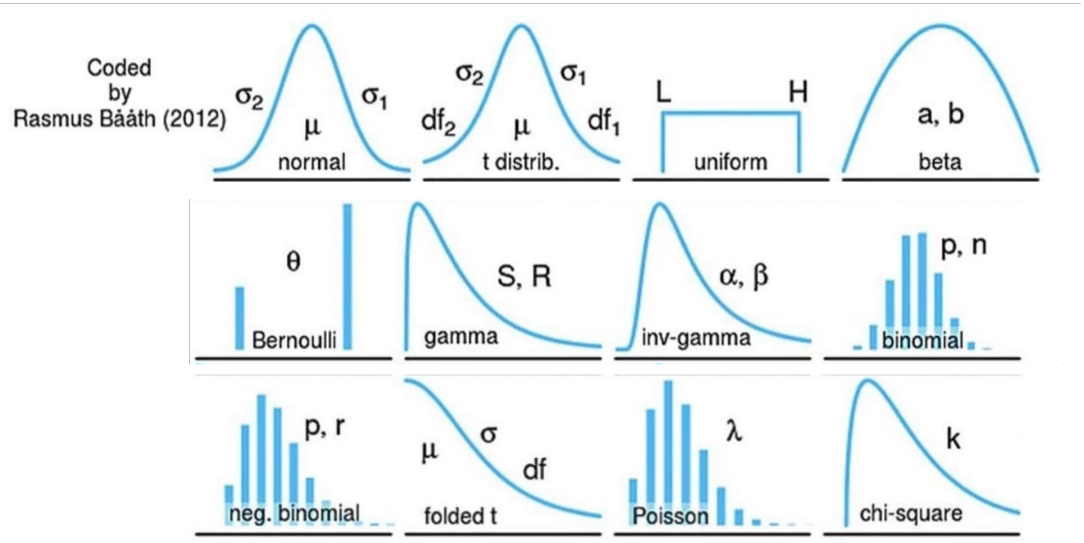
In practice, many forms of data do not meet this assumption.

- In some cases, you can transform the variable (e.g., log transformation to reduce skewness) to make the data more normally distributed (e.g., income).
- In other cases, you will need to use a model that assumes that your outcome comes from a non-Gaussian distribution family.

Generalized Linear Models (GLMs)

GLMs are used with data types like binary variables, counts, and proportions.

The outcome variable typically comes from a non-normal/Gaussian distribution, like the binomial (e.g., Bernoulli for binary variables), Poisson (e.g., for counts), beta (e.g., for proportions) and so on.



Generalized Linear Models (GLMs)

Link functions connect (or “link”) the linear predictors to the expected value of the outcome variable’s distribution, often by transforming the outcome to a scale where linear modeling is appropriate.

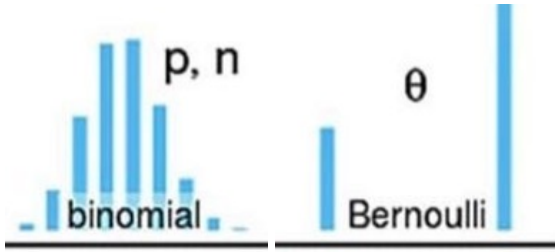
Distribution	Common Link Function	Purpose/Usage	Formula
Normal	Identity	Directly relates the linear predictor to the response variable. Used for continuous data.	$\eta = \mu$
Binomial	Logit	Transforms the probability of success to an unbounded scale. Used for binary outcomes (e.g., success/failure).	$\eta = \log\left(\frac{\mu}{1-\mu}\right)$
Poisson	Log	Relates the log of the mean count to the linear predictors. Used for count data.	$\eta = \log(\mu)$

Logistic Regression

When to use? When your outcome variable is binary (e.g., yes/no, true/false, case/control, diagnosis A vs diagnosis B).

Distribution: binomial distribution.

Link function: logit (log-odds).



The coefficients/estimates from the model output are logits and are not easily interpretable, but can be converted to odds ratios: e^{logit}.

- Odds ratios indicate the odds of your outcome variable occurring as you increase your predictor by 1 unit (for continuous variables) or go from one group to the next (for categorical variables).

Logistic Regression

Let’s look at the relationship between smoking status, sex, and heart disease.

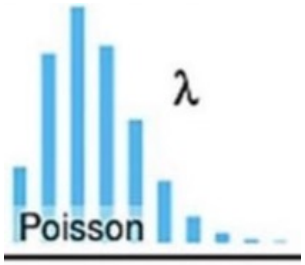
```
log_model <- glm(TenYearCHD ~ sex * currentSmoker, data = df, family =
binomial(link=“logit”))

exp(coef(log_model))
```

Poisson Regression

When to use? When your outcome variable is a count and the mean is approximately equal to the variance. This approach is especially suitable for rare events (e.g., fewer than 10 counts).

Distribution: Poisson.

Link function: natural log (ln).



The coefficients/estimates from the model output are on the log scale and represent the log of the expected count. They can be exponentiated to obtain rate ratios: e^{β}.

Poisson Regression

Let’s look at the relationship between lifestyle factors and physiological factors.

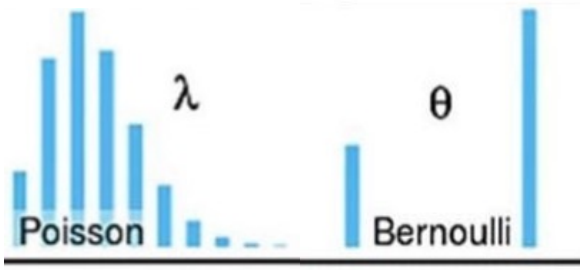
```
poisson_model <- glm(biometric_flags ~ age * sex + education +
cigsPerDay + diabetes, data = df, family = poisson(link="log"))

exp(coef(poisson_model))
```

Zero-Inflated Poisson Regression

When to use? When your outcome variable is a count, but there are more 0's in your data than expected.

Distribution: Poisson, Bernoulli.

Link function: log, logit.



The zero-inflated Poisson (ZIP) model assumes that zero counts can arise from:

1. A Bernoulli process where a datapoint can belong to the always-zero group or not.
2. A Poisson process where a datapoint can take on a value of zero or a count.

Zero-Inflated Poisson Regression

Let's look at the relationship between smoking status, sex, and heart disease.

```
zip_model <- zeroinfl(cigsPerDay ~ age * sex | age * sex,
                      data = df,
                      dist = "poisson")
```

The first `age * sex` term models the Poisson process (counts) and the second `age * sex` term models the Bernoulli process (zero-inflation).

Other GLMs

Negative binomial models

- Generalization of Poisson regression, when the count data are over-dispersed i.e., their variance is greater than the mean. Uses the log link function and coefficients represent the log of expected counts. Coefficients can be exponentiated to get rate ratios.

Multinomial regression

- Generalization of logistic regression, when there are >2 unordered categories in the outcome variable. Uses the generalized logit link function and coefficients represent the change in log-odds of being in one category compared to the reference category. As with logistic regression, coefficients can be exponentiated to get odds ratios.

Ordinal regression

- Generalization of logistic regression, when there are >2 ordered categories in the outcome variable. Uses the cumulative logit link function and coefficients represent the change in log-odds of being in a higher vs lower category. Coefficients can be exponentiated to get cumulative odds ratios.

Beta regression

- Used when the outcome variable is a proportion between 0 and 1, exclusive. Uses a logit or log link function and coefficients represent the change in the mean proportion.